

New Models, Services, and Frameworks

Claude 3.7 Sonnet with reasoning capabilities

<https://www.anthropic.com/news/claude-3-7-sonnet>

See also <https://www.anthropic.com/research/visible-extended-thinking>

System prompts: <https://docs.anthropic.com/en/release-notes/system-prompts>

The long-awaited reasoning model by Anthropic!

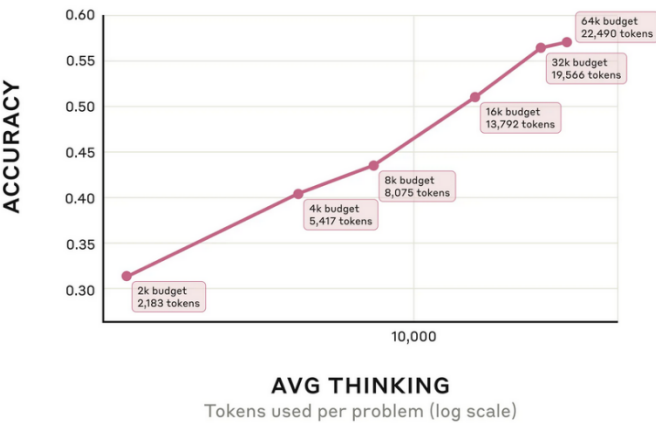
Some interesting things about it are reasoning transparency and the possibility of controlling the reasoning length when calling the API (see here

<https://docs.anthropic.com/en/docs/build-with-claude/extended-thinking> for more details). This can allow you to tune the trade off between quality and cost. (Which taking into account the fact that Claude is the most expensive model on the market is an important thing!) On the right you can see an illustration for the AIME math Olympiad dataset.

The new model seems to become even better at coding, and also at agentic tasks (check also the Overthinking paper’s review about Claude vs other models in agentic tasks)

AIME 2024 performance

vs. actual thinking token usage



	Claude 3.7 Sonnet 64K extended thinking	Claude 3.7 Sonnet No extended thinking	Claude 3.5 Sonnet (new)	OpenAI o1 ¹	OpenAI o3-mini ¹ High	DeepSeek R1 32K extended thinking	Grok 3 Beta Extended thinking
Graduate-level reasoning <i>GPQA Diamond³</i>	78.2% / 84.8%	68.0%	65.0%	75.7% / 78.0%	79.7%	71.5%	80.2% / 84.6%
Agentic coding <i>SWE-bench Verified²</i>	—	62.3% / 70.3%	49.0%	48.9%	49.3%	49.2%	—
Agentic tool use <i>TAU-bench</i>	—	Retail 81.2%	Retail 71.5%	Retail 73.5%	—	—	—
	—	Airline 58.4%	Airline 48.8%	Airline 54.2%	—	—	—
Multilingual Q&A <i>MMMLU</i>	86.1%	83.2%	82.1%	87.7%	79.5%	—	—
Visual reasoning <i>MMMU (validation)</i>	75%	71.8%	70.4%	78.2%	—	—	76.0% / 78.0%
Instruction-following <i>IFEval</i>	93.2%	90.8%	90.2%	—	—	83.3%	—
Math problem-solving <i>MATH 500</i>	96.2%	82.2%	78.0%	96.4%	97.9%	97.3%	—
High school math competition <i>AIME 2024⁴</i>	61.3% / 80.0%	23.3%	16.0%	79.2% / 83.3%	87.3%	79.8%	83.9% / 93.3%

MLGym: A New Framework and Benchmark for Advancing AI Research Agents (by Meta)

<https://arxiv.org/pdf/2502.14499>

Some demos: <https://sites.google.com/view/mlgym>

Github: <https://github.com/facebookresearch/mlgym>

When I studied Reinforcement Learning back in 2017, our practice heavily relied on the Gym framework which allowed to interact conveniently with several RL environments (like Cart Pole, Lunar Landar, Atari games etc) and even create new ones. Probably, MLGym will play a similar role for LLM Agents and RL fine tuning of LLMs for agentic tasks.

MLGym brings together LLMs and tools (including agent-computer interaction) that can be launched in a safe environment. It also has a number of tools for agent evaluation.

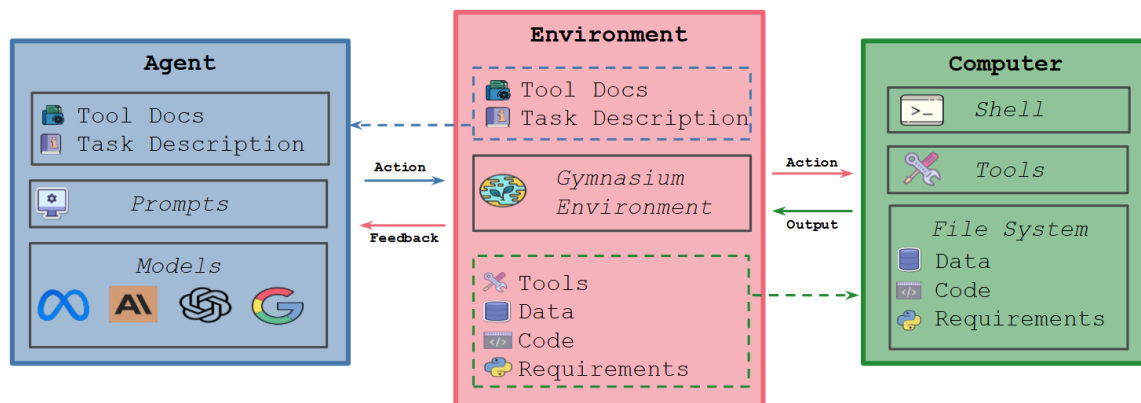


Figure 1 Diagram of MLGYM, a unified framework designed to integrate diverse and open-ended AI research tasks into a single platform for developing and evaluating LLM agents on these tasks.

Meta team evaluated agents based on different LLMs on a number of very diverse tasks, including:

- Prisoners Dilemma (finding a strategy)
- Training a GPT-2 style model in FineWeb (the agent has a baseline code, it needs to optimize validation loss) <https://www.youtube.com/watch?v=yiinpl7Vmzg>
- Classifying images from CIFAR-10 <https://www.youtube.com/watch?v=kp27HTk1FOw>

These evaluation efforts allow the researchers to create the following comparison which is surprisingly favourable for Gemini:



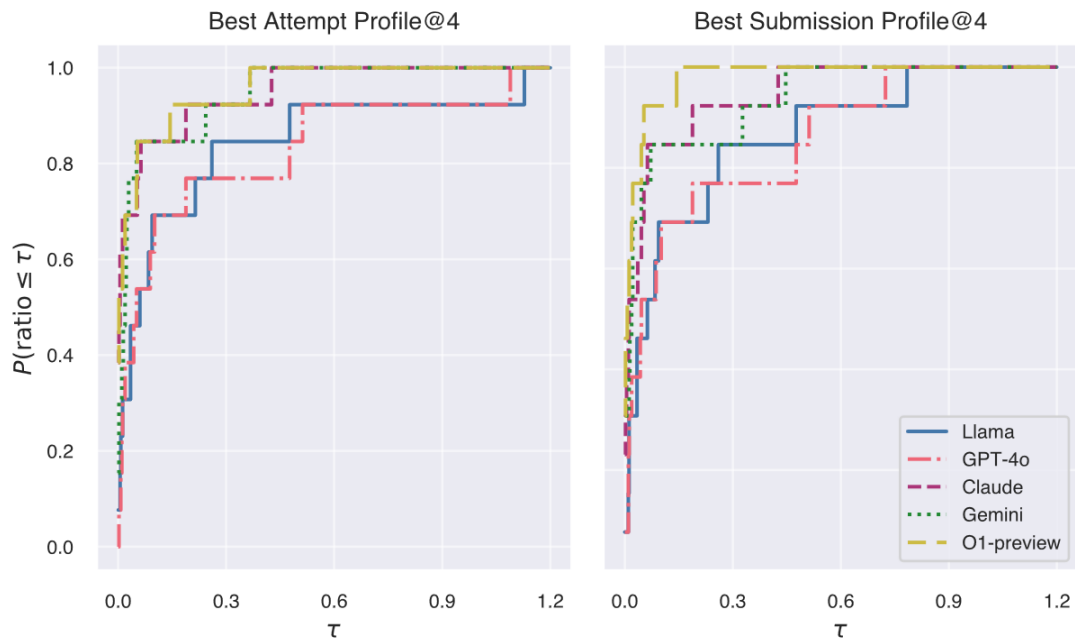
If you're not very curious about AUP@4 (the metric on the y axis), then just keep in mind that its larger values are better. If you are, I'll briefly explain what it is. Imagine that we have a task t whose metric is *the less the better* (like $1 - \text{accuracy}$ or MSE). Then for a model m we compute

$$r_{t,m} = \frac{\text{metric on } t \text{ of an agent based on } m}{\text{best metric on } t \text{ among all models}}$$

Now, having this, we further compute for a threshold τ :

$$\rho_m(\tau) = \frac{\text{On how many tasks } \log_{10} r_{t,m} \leq \tau}{\text{Number of tasks}}$$

Given that the metrics are *the less the better*, this number shows the ratio of tasks where m performs only the given times worse than the best model. Calculating these values for different τ , we get the **performance profile curve**.



The “@4” suffix shows that we take the best metric values over 4 runs. **AUP is the area under this curve.**

Of course, a very strong temptation is trying to use such agents in research tasks, to create some novel (?) scientific results. But after testing MLGym, the authors conclude that LLM agents are still at the level

Level 1: Baseline Improvement At Level 1, LLM agents can improve performance on a benchmark given a baseline code that is not state-of-the-art (SOTA). This level indicates the ability to analyze and optimize existing solutions, even if they are not the most advanced.

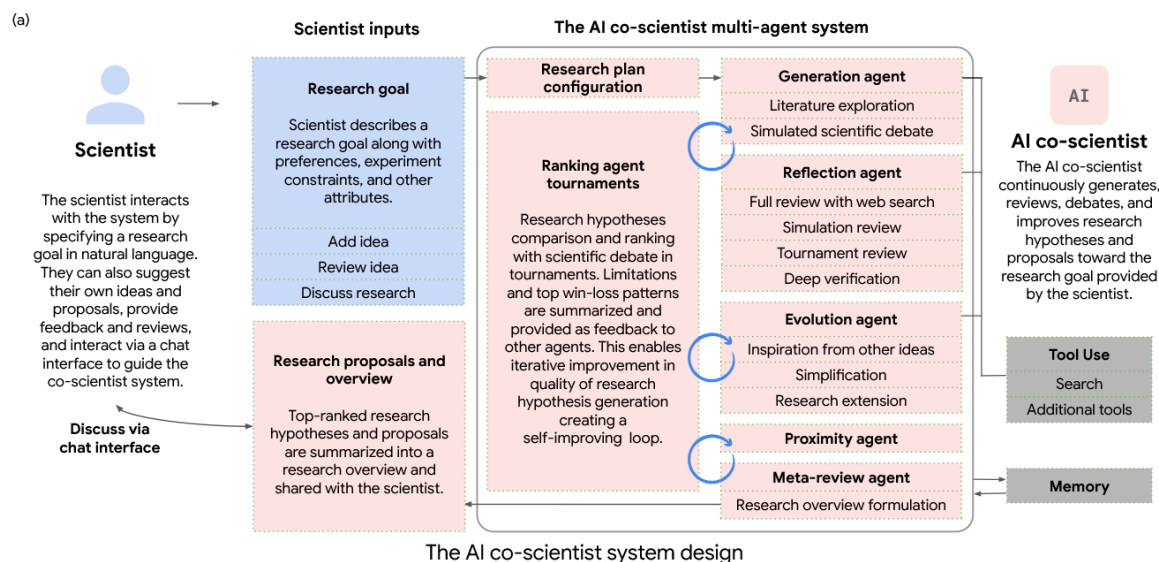
So, no LLM-written papers at NeurIPS :) At least, I hope so... Given the worsening quality of peer-reviews and somewhat lowering expectations from the research papers in AI, well...

Towards and AI co-scientist (by Google)

https://storage.googleapis.com/coscientist_paper/ai_coscientist.pdf

Let’s go on with the research topic. Since LLMs aren’t at the point of being self-sufficient researchers now, probably they can be research assistants?

This paper by Google investigates it and comes up with the AI co-scientist framework which is able to suggest hypotheses for an actual researcher to check.



Of course, this is an agentic framework, and quite an elaborate one. It has several components:

- **Generation agent** explores relevant literature using web search and formulates new research directions.
- **Reflection agent** simulates the role of a scientific peer reviewer, critically evaluating each hypothesis.

- **Ranking agent.** That's quite an interesting one! It orchestrates an Elo-based "tournament" to assess and prioritize the generated hypotheses. This involves pairwise comparisons, facilitated by simulated scientific debates.
- **Proximity agent** computes a proximity graph for generated hypotheses, enabling clustering of similar ideas, de-duplication etc.
- **Evolution agent** continuously refines the top-ranked hypotheses emerging from the tournament. Its refinement strategies include synthesizing existing ideas, using analogies, leveraging literature for supporting details, exploring unconventional reasoning, and simplifying concepts for clarity.
- **Meta-review agent** synthesizes insights from all reviews, identifying recurring patterns in tournament debates, and using these findings to optimize other agents' performance in subsequent iterations. It agent synthesizes top-ranked hypotheses and reviews into a comprehensive research overview for review by the scientist.

This framework was tested in two biotech research areas and yielded exciting results. In some cases it suggested a hypothesis which scientists became excited to check in a lab, and in another case it rediscovered a hypothesis which has already been proved fruitful, but the framework could get no information about it.

Grok 3

Try it here if you have an X account: <https://x.com/i/grok>

The recent release by X; has Deep Research and Thinking capabilities.

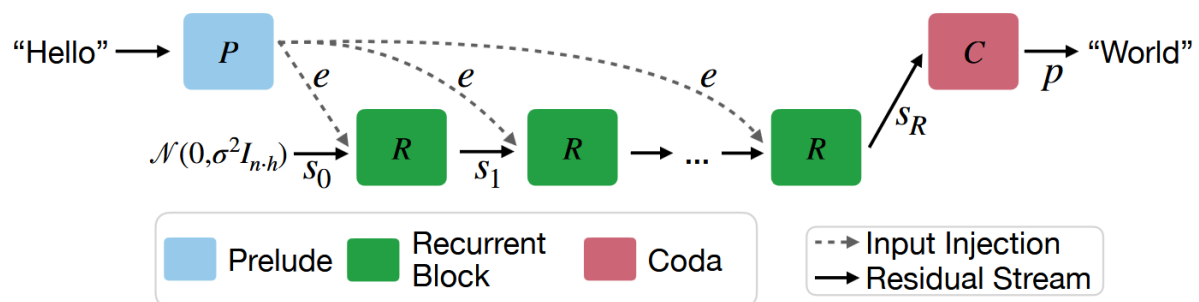
The release itself seems half-baked, with little information about benchmarks and safety. I tried it. To tell the truth, so far, I liked Deep Research from Gemini more (and I look forward to trying OpenAI's Deep Research when they finally make it available for Plus users!) Reasoning looks ok to me.

Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach

<https://arxiv.org/pdf/2502.05171>

That's one more work that tries to leverage inference-time compute in pursuit of making LLMs smarter. To me, it much resembles [Quiet-STaR](#), but has its own interesting ideas.

The idea is to plug between each pair of tokens a recurrent subnetwork, which would perform covert reasoning in latent space:



Previous token is also used in every recurrent block to stabilize the procedure.

The whole network is trained from scratch with different recurrent subsequence lengths (see the length distribution on the right).

During inference, by default the number of recurrent iterations may be

- Constant
- Data-dependent, with recurrence finishing when converging to a certain vector
- Set small as a way of using native speculative decoding

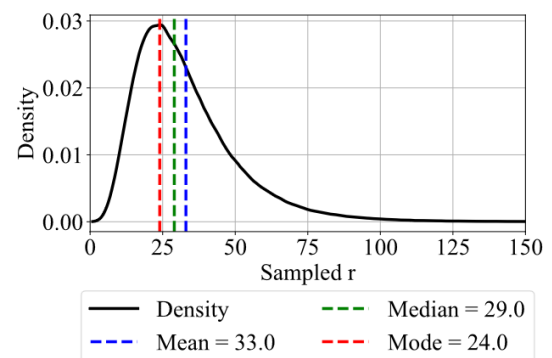
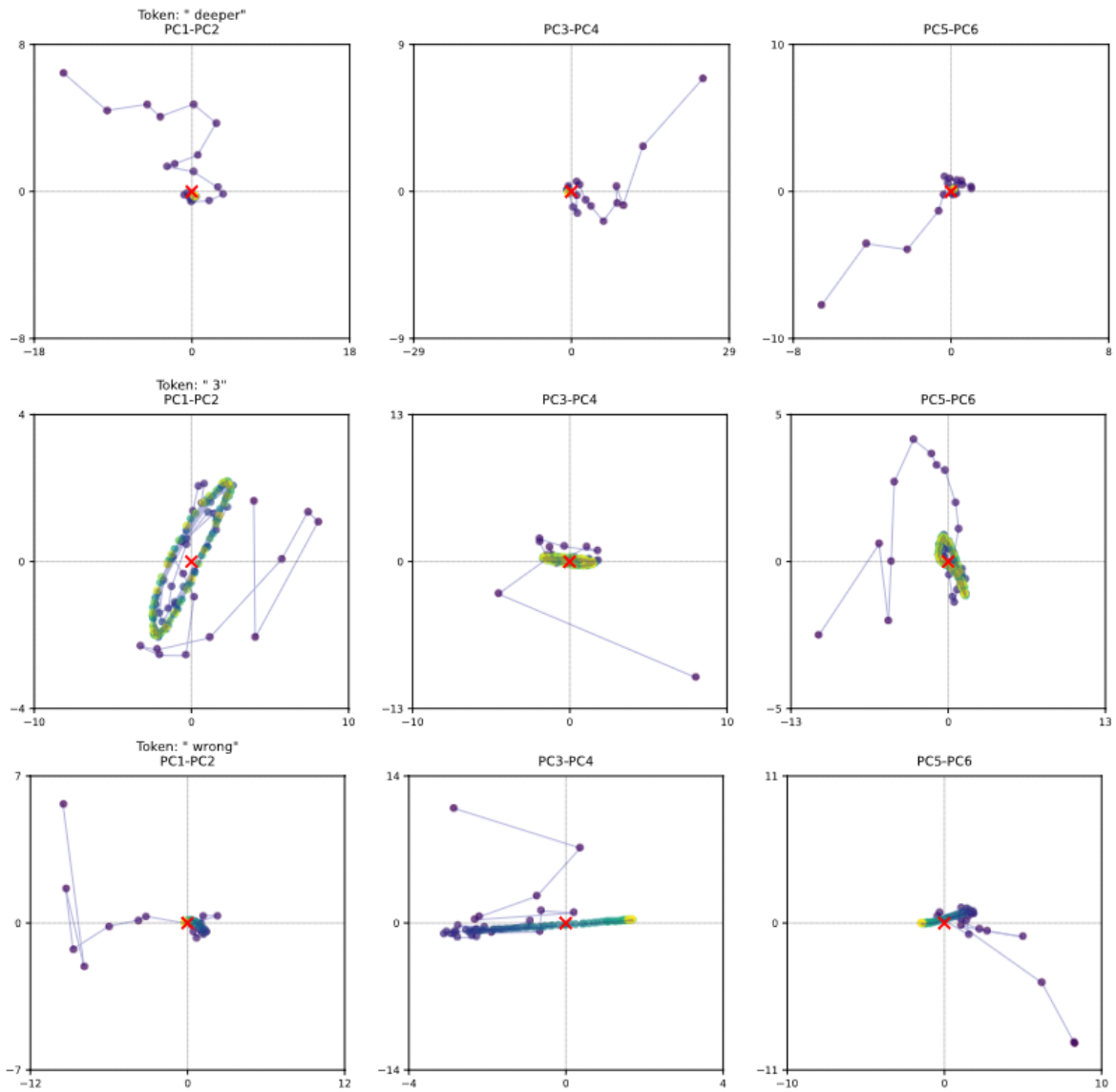


Figure 3: We use a log-normal Poisson Distribution to sample the number of recurrent iterations for each training step.

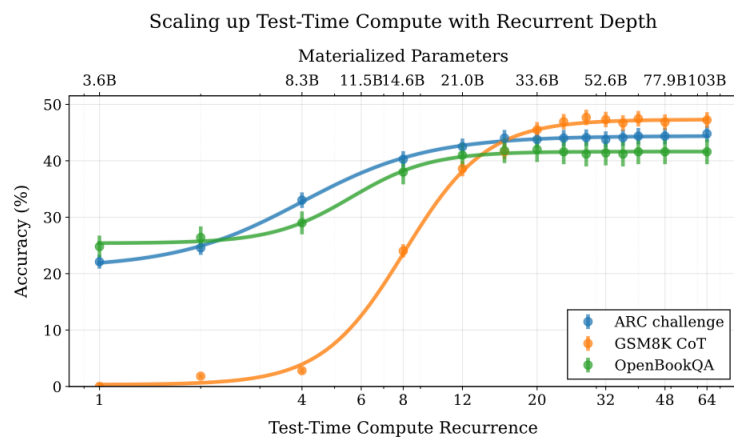
The authors investigate in more details the patterns of the recurrent process convergence, and they find that there are several of them (see the picture on the next page):

- Just convergence to a point (top row)
- Convergence to a terminal cycle (middle row)
- Convergence to a "slider" (bottom row)

There are reasons to suggest that the last two may represent some advanced concepts like arithmetic (though maybe not).



The model is only compared with some of the weaker LLMs, so it's not very interesting, and I'll finish just with a plot showing how accuracy changes with recurrence length.



LIMO: Less is More for Reasoning <https://arxiv.org/pdf/2502.03387>

This paper demonstrates that through training on only **817** high-quality curated training samples an LLM can reach quite impressive math performance with exceptional out-of-distribution generalization. (With an important addition that all the necessary knowledge should already be there after pre-training). I'd add that this resonates with the [Demystifying Long Chain-of-Thought Reasoning in LLMs](#) paper's claim that **Core reasoning abilities like error correction are inherently present in base models, even if activating them may be tricky**. I'd add that this was probably what made DeepSeek R1Zero possible. A little reminder: R1Zero was finetuned from a pre-trained DeepSeek V3 with only RL, and moreover, the reward was answer + format, no special reasoning supervision – still, R1Zero learnt to reason in a non-linear way. How? Well, not many options but to suppose that V3 “saw” some non-linear reasoning during the training, and RL just activated this latent capability.

Ok, but let's briefly discuss LIMO. It doesn't use RL. Instead, it's trained via honest SFT on (problem, solution) pairs. It wasn't an easy task to choose just 817 examples from much larger datasets and generate high-quality solutions; while doing this, the authors came up with some guidelines for what to consider a high-quality solution for SFT. I think, it's interesting to look at the criteria:

- **Structured Organization:** The solution is well-organized, adjusting detail based on step complexity—allocating more tokens for key reasoning points while keeping simpler steps concise.
- **Cognitive Scaffolding:** It builds understanding progressively, introducing concepts strategically and bridging gaps to make complex reasoning more accessible.
- **Rigorous Verification:** Frequent checks validate intermediate results, cross-check assumptions, and ensure logical consistency for a reliable final answer.

The results are really impressive. And it takes some courage to compare your model with o1-preview.

Datasets	OpenAI-o1-preview	Qwen2.5-32B-Instruct	QwQ-32B-preview	OpenThoughts (114k)	NuminaMath (100k)	LIMO ours(817)
In Domain						
AIME24	44.6	16.5	50.0	50.2	6.5	57.1
MATH500	85.5	79.4	89.8	80.6	59.2	94.8
AMC23	81.8	64.0	83.6	80.5	40.6	92.0
Out of Domain						
OlympiadBench	52.1	45.3	58.5	56.3	36.7	66.8
CHMath	50.0	27.3	68.5	74.1	11.2	75.4
Gaokao	62.1	72.1	80.1	63.2	49.4	81.0
Kaoyan	51.5	48.2	70.3	54.7	32.7	73.4
GradeSchool	62.8	56.7	63.8	39.0	36.2	76.2
Minerva	47.1	41.2	39.0	41.1	24.6	44.9
GPQA	73.3	48.0	65.1	42.9	25.8	66.7
AVG.	61.1	49.9	66.9	58.3	32.3	72.8

The Danger of Overthinking: Examining the Reasoning-Action Dilemma in Agentic Tasks

<https://arxiv.org/pdf/2502.08235>

Two weeks ago, we discussed [the perils of underthinking](#); now it's our time to observe that overthinking is also a problem!

This paper discusses how LLMs perform in agentic tasks, and they define **overthinking** as the tendency of an LRM to rely excessively on internal reasoning instead of interacting with the environment and gathering feedback from it. (Like theorizing instead of experimenting.)

And overthinking seems to be harmful for agentic capabilities (see the plots to the right, showing different patterns for non-reasoning and reasoning models)

The researchers identified several potential manifestations of overthinking:

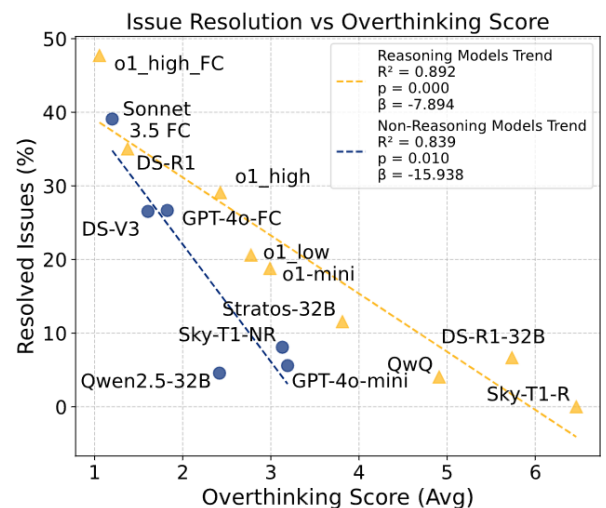


Figure 1. Higher overthinking scores (tendency to favor internal reasoning over environmental feedback) correlate with lower issue resolution rates across all models. Reasoning models exhibit consistently higher overthinking tendencies, suggesting that excessive reliance on internal simulation impairs task performance. Model nomenclature: "FC" indicates native function calling capability, "DS" represents DeepSeek models, and suffixes o1_high and o1_low denote models with reasoning effort set to high and low respectively.

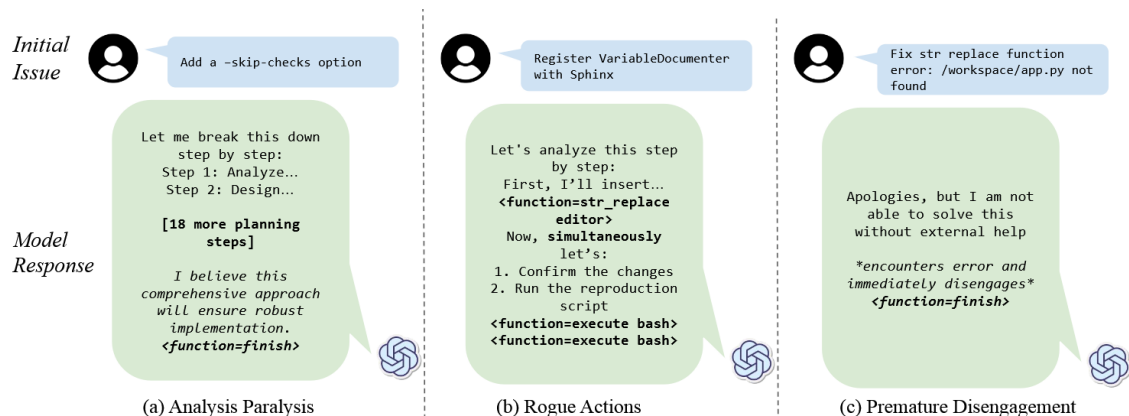


Figure 4. Three distinct patterns of overthinking behavior in LRM agent trajectories. (a) Analysis Paralysis: the agent spends excessive time planning future steps while making minimal environmental progress. (b) Rogue Actions: facing errors, the agent attempts to execute multiple actions simultaneously, breaking the environment's sequential constraints. (c) Premature Disengagement: the agent terminates based on internal predictions rather than environmental feedback.

- **Analysis Paralysis:** elaborate future planning while failing to execute the plans
- **Rogue Actions:** an agent generates chains of interdependent actions in a single step, without awaiting feedback from the environment, "assuming" from the beginning that each step is successful and effectively substituting real

environmental feedback with internal simulation (sounds a bit like running multiple cells in colab not taking into account that the first one may fail).

- **Premature Disengagement:** terminating tasks based solely on their internal simulation of the problem space, either through direct abandonment or by delegating hypothetical action sequence. (In the latter case, effectively giving an unchecked solution).

The authors decide to measure overthinking in quite a naïve way: by prompting an LLM to count the three above events and their severity, eventually giving the answer on a 0-10 scale.

It's not clear what to do with overthinking. You can choose a solution with the lowest overthinking score and hope that it will be of the best quality – but for that you need to generate several solutions, which is a whole lot of trouble anyway. However, it may be beneficial to check general overthinking score of an LLM before using it in agentic tasks. Quite curiously, by the way, DeepSeek R1 exhibits low overthinking.

Let's finish the tale about this paper with my experiments :) When Claude 3.7 Sonnet with thinking capabilities finally became available for me this morning, I decided to give some task (the same one) to all the reasoning LLMs I know. And since I was somewhat tired of probing LLMs with math, I asked them the following:

I want you to navigate the following maze, where spaces indicate a clear path and symbols "=", "|", ".", and "+" are walls. Here, S stands for start and E stands for exit. Your final output should be the map of the labyrinth with your trajectory shown in ""*

```

. . .

=====

S      |      |      |
.===.  '= .  .== |  '====== |
|      |      |      |      | | | |
|  ==+==  |= '  .=====|
|      |      |      |      |
|==  |  ==|  =='  ==.  |  .==  |
|      |      |      |      |
|  |  |==  |  ===== '  |  |  |
|  |  |      |      |      |  |  |

```

```

| '=| =====| =====' | | | |
|   |           |   |   | | |
| == '=====, | | | ,.= ' | |
|           | | |   |   | |
| | ,.===== ' | '===| =====|
| | |           |   | |   |
| '=' ,.=====, | |===== |
|           |   | |   |
| =====+= ,.=, | | | =====|
|           | | |   |   |
| ===, | ==| | |== '===== |
|   | |   |   |   |   |
| == ' '== | ====== ' | |
|           |           | E
'=====
...

```

And it turned out that the results somehow align with what was observed in the paper (I used playgrounds for all the models). Don't take this experiment too seriously, please 😊. But it sounds fun.

- **o3-mini-high** spent 6 minutes on philosophy, complaining that “The maze is 25x25, making it too complex to solve manually.” and finally outputting

I think I can manually trace a path through the maze by converting spaces along a plausible trajectory into "". A potential path could start at S (row 2, col 1), moving right along row 2 to col 6. Then, I could move left, right, and down step by step across the maze, sticking to the open paths. I'll design a continuous line from S to E while respecting the walls, marking each valid step with "". However, it's hard to guarantee perfection without a solver!

So yes, it gave up.

- **Claude 3.7 Sonnet** just wrote the Python code and executed it producing the valid path (that's the winner!). In a sense, it is a cheat (indeed, Claude can run code in the playground), but this showcases lack of overthinking.
- **Grok 3** was actually not bad. It spent 7 minutes outputting things like

Wait, row 20: "| | | | | |", likely (20,1): "|", (20,2 to 20,6): " ", (20,7): "|", (20,8 to 20,10): " ", (20,11): "|", (20,12): " ", (20,13): "|", (20,14): " ", (20,15): "|", (20,16): " ", (20,17 to 20,21): " ", (20,22): "|", (20,23 to 20,25): " ", but since it's 25 characters, perhaps (20,23): " ", (20,24): " ", (20,25): "|".

The trajectory it suggested in the end had bugs after its first 1/3, but still, it was a sincere effort.

- **Gemini** cheated. It suggested to code and quite vaguely explained that it wanted to code DFS (no code provided). Everything ended with

Let's start coding. First, parse the input maze string to 2D array. Find S and E positions. Then implement DFS. Finally, format the output.

and an incorrect path mapping with *. That really looked like Premature Disengagement to me.

- **DeepSeek R1**, sadly, didn't live up to my expectations. There was much philosophy before it actually attempted to do BFS textually, and these attempts were rather weak, resulting in

But given the time constraints, I'll proceed to provide the final maze with the path marked as per the BFS algorithm.

and an incorrect partial solution :(

